



학습 목표

1. 소프트웨어의 중요성을 안다.
2. C++ 언어의 역사를 이해한다.
3. C++ 언어의 특징을 이해한다
4. C++ 프로그램의 개발 과정을 안다.
5. C++ 표준 라이브러리에 대해 안다.

세상을 먹어 치우는 소프트웨어

3

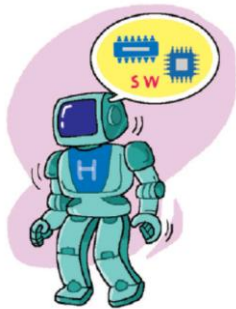
□ 소프트웨어 기업이 세상을 지배

Software is eating the world.

eBay, Facebook, Groupon, Skype, Twitter, Android, Netflix, Google, Apple, Samsung

□ 4차 산업의 핵심에 소프트웨어가 있다

- ▣ 인공지능, 빅데이터, 자율주행 자동차, 반도체, 로봇 등



소프트웨어와 컴퓨터

4

- 컴퓨터(Computer)
 - ▣ 정보 처리 장치
 - ▣ 메인 프레임, PC, 태블릿, 스마트폰, One-chip 컴퓨터 등
- 하드웨어(Hardware)
 - ▣ 눈에 보이고 만질 수 있는 물리적인 컴퓨터 부품(기계 장치)
 - ▣ CPU, Memory, I/O Device 등
- 소프트웨어(Software)
 - ▣ 컴퓨터 하드웨어를 작동시켜 웹서핑, 음악듣기, 영화보기 등을 수행하는 프로그램
 - ▣ CPU가 이해할 수 있는 일련의 명령어들과 데이터로 구성됨
 - ▣ CPU는 이 명령들을 순차적으로 해석하여 실행한다.
 - ▣ 제품의 경쟁력을 좌우하는 가장 중요한 요소

프로그래밍과 프로그래밍 언어

5

□ 프로그래밍

- ▣ 컴퓨터가 처리할 일련의 작업을 작성하는 것

□ 프로그래밍 언어

▣ 기계어(machine language)

- 0, 1의 이진수로 구성된 언어
- 컴퓨터의 CPU는 본질적으로 기계어만 처리 가능

▣ 어셈블리어

- 기계어의 명령을 ADD, SUB, MOVE 등과 같은 상징적인 니모닉 기호(mnemonic symbol)로 일대일 대응시킨 언어
- 어셈블러 : 어셈블리어 프로그램을 기계어 코드로 변환

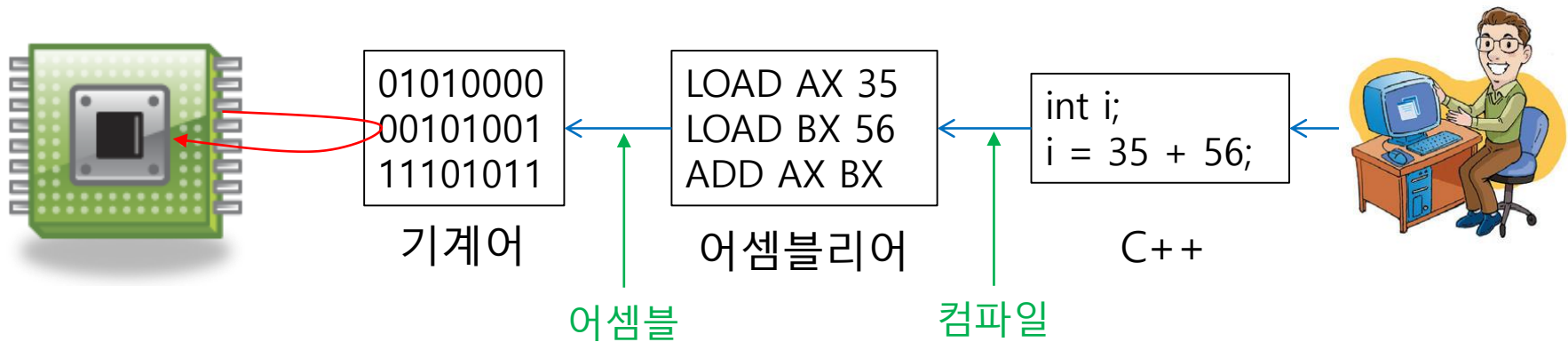
▣ 고급언어

- 사람이 이해하기 쉽고 복잡한 알고리즘을 표현하기 위해 고안된 언어
- Pascal, Basic, C/C++, Java, C#, Python
- 컴파일러 : 고급 언어로 작성된 프로그램을 기계어 코드로 변환

사람과 컴퓨터, 기계어와 고급 언어

6

$$35 + 56 = ?$$



7

Martin Richards

Ken Thompson

Ken Thompson과
Dennis Ritchie



표준 C++ 프로그램의 중요성

8

□ C++ 언어의 표준

- ▣ 1998년 미국 표준원(ANSI, American National Standards Institute)에서 C++ 언어에 대한 표준 정립
- ▣ ISO/IEC 14882 문서에 작성됨. 유료 문서
- ▣ 표준의 진화
 - 1998년(C++98), 2003년(C++03), 2007년(C++TR1), 2011년(C++11)

□ 표준의 중요성

- ▣ 표준에 따라 작성된 C++ 프로그램은 모든 C++ 컴파일러에 의해 컴파일 가능하고 동일한 실행 결과 보장

□ 비표준 C++ 문법

- ▣ Clang C++, Visual C++, 등 대부분의 컴파일러는 C++표준에 자신만의 기능을 추가
- ▣ 특정 C++ 컴파일러에서만 컴파일되므로 호환성 결여

표준/비표준 C++ 프로그램의 비교

9

표준 C++ 규칙에 따라
작성된 C++ 프로그램

```
#include <iostream>
int main() {
    std::cout << "Hello";
    return 0;
}
```

모든 C++
컴파일러
에 의해
컴파일

볼랜드 C++
컴파일러

비주얼 C++
컴파일러

GNU C++
컴파일러

실행
파일

실행
파일

실행
파일

컴퓨터

표준 C++ 규칙에 따라
작성되지 않는 비주얼 C++ 프로그램

```
#include <iostream>
int _cdecl main() {
    std::cout << "Hello";
    return 0;
}
```

비주얼
C++ 전용
키워드

clang C++
컴파일러

비주얼 C++
컴파일러

GNU C++
컴파일러

~~실행
파일~~

실행
파일

~~실행
파일~~

컴퓨터

C++ 언어의 주요한 특징

10

- C 언어와의 호환성
 - ▣ C 언어의 문법 체계 계승
 - 소스 레벨 호환성 - 기존에 작성된 C 소스파일을 그대로 가져다 사용
 - 링크 레벨 호환성 - C 라이브러리(목적파일)를 C++ 프로그램에서 링크
- 객체 지향 개념 도입
 - ▣ 캡슐화, 상속, 다형성
 - ▣ 소프트웨어의 재사용을 통해 생산성 향상
 - ▣ 복잡하고 큰 규모의 소프트웨어의 작성, 관리, 유지보수 용이
- 엄격한 타입 체크
 - ▣ 실행 시간 오류의 가능성을 줄임
 - ▣ 디버깅 편리
- 실행 시간의 효율성 저하 최소화
 - ▣ 실행 시간을 저하시키는 요소와 해결
 - 작은 크기의 멤버 함수 잦은 호출 가능성 -> 인라인 함수로 실행 시간 저하 해소

C 언어에 추가한 기능

11

- 함수 중복(function overloading)
 - ▣ 매개 변수의 개수나 타입이 다른 동일한 이름의 함수들 선언
- 디폴트 매개 변수(default parameter)
 - ▣ 매개 변수에 디폴트 값이 전달되도록 함수 선언
- 참조와 참조 변수(reference)
 - ▣ 하나의 변수에 별명을 사용하는 참조 변수 도입
- 참조에 의한 호출(call-by-reference)
 - ▣ 함수 호출 시 참조 전달
- new/delete 연산자
 - ▣ 동적 메모리 할당/해제를 위해 new와 delete 연산자 도입
- 연산자 재정의
 - ▣ 기존 C++ 연산자에 새로운 연산 정의
- 제네릭 함수와 제네릭 클래스
 - ▣ 데이터 타입에 의존하지 않고 일반화시킨 함수나 클래스 작성 가능

C++ 객체 지향 특성 - 캡슐화

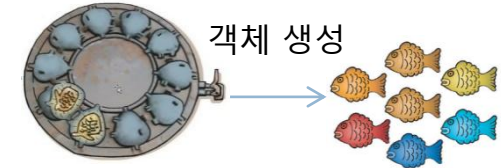
12

□ 캡슐화(Encapsulation)

- ▣ 데이터를 캡슐로 싸서 외부 접근으로부터 보호
- ▣ C++에서 클래스가 캡슐 역할을 함

□ 클래스와 객체

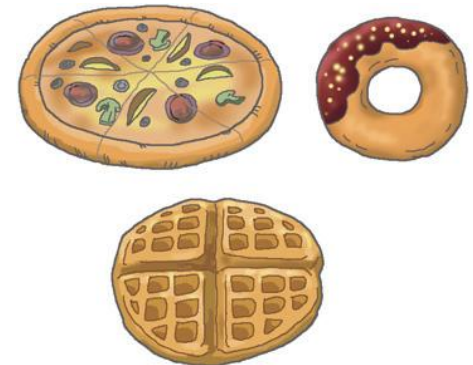
- ▣ 클래스 - 객체를 만드는 틀(**사용자 정의 자료형**)
- ▣ 객체 - 클래스라는 틀에서 생겨난 실체(**클래스형 변수**)
- ▣ 객체(object), 실체(instance)는 같은 뜻



클래스((객체를 만드는 틀) 객체들(실체)

```
class Circle {  
private:  
    int radius; // 반지름 값  
public:  
    Circle(int r) { radius = r; }  
    double getArea() { return 3.14 * radius * radius; }  
};
```

원을 추상화한 Circle 클래스



원 객체들(실체)

C++ 객체 지향 특성 - 상속성

13

□ 상속성(Inheritance)

- 자식이 부모의 유전자를 물려 받는 것과 유사-> 기존 코드를 쉽고 효율적으로 재활용하는 기능
- 객체가 생성될 때 자식 클래스의 멤버 뿐 아니라 부모 클래스 멤버들을 함께 가지고 탄생



```
class Phone {  
    void call();  
    void receive();  
};  
  
class MobilePhone : public Phone {  
    void connectWireless();  
    void recharge();  
};  
  
class MusicPhone : public MobilePhone {  
    void downloadMusic();  
    void play();  
};
```

C++로 상속 선언



전화기



휴대 전화기



음악 가능
전화기

C++ 객체 지향 특성 - 다형성

14

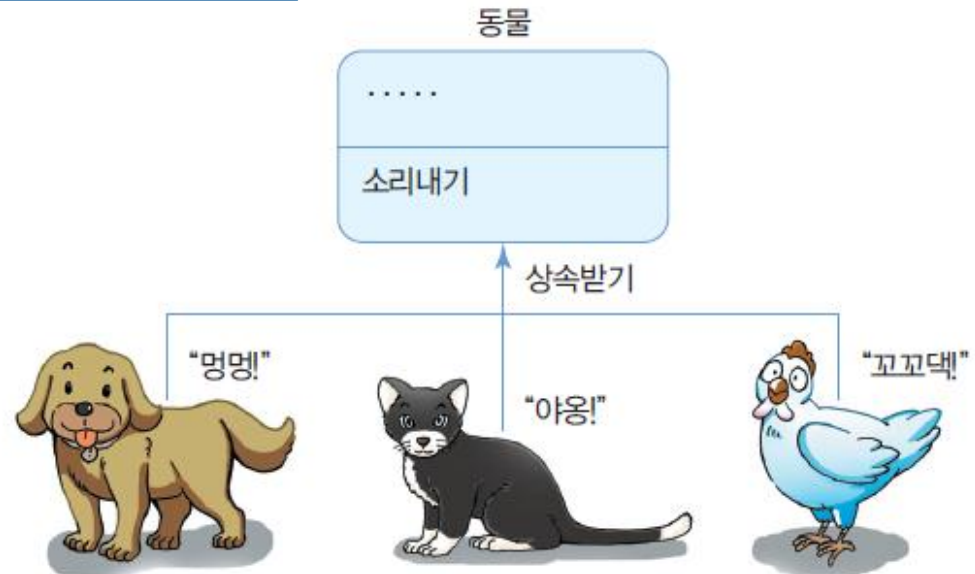
- 다형성(Polymorphism)
 - ▣ 하나의 기능(함수)이 상황에 따라 다르게 동작하게 하는 기술
 - ▣ 연산자 중복, 함수 중복, 함수 재정의(overriding)

```
2 + 3    --> 5  
"남자" + "여자" --> "남자여자"  
redColor 객체 + blueColor 객체 --> purpleColor 객체
```

+ 연산자 중복

```
void add(int a, int b) { ... }  
void add(int a, int b, int c) { ... }  
void add(int a, double d) { ... }
```

add 함수 중복



함수 재정의(오버라이딩)

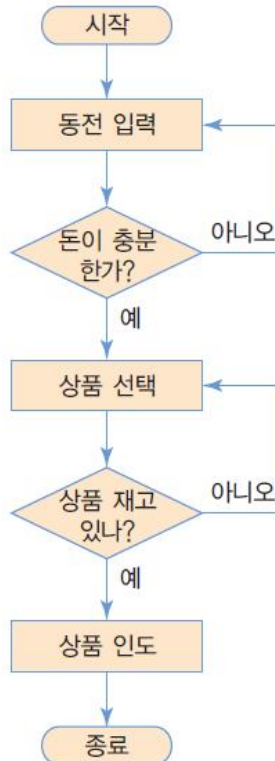
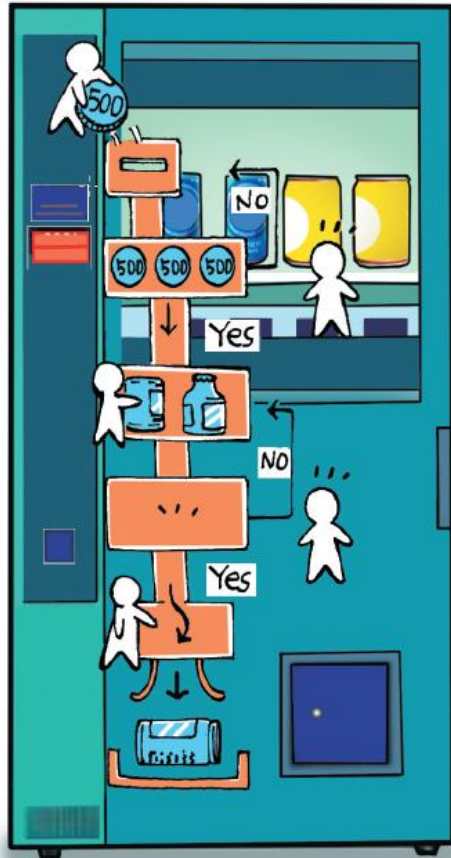
C++ 언어에서 객체 지향을 도입한 목적

15

- 소프트웨어 생산성 향상
 - ▣ 소프트웨어의 유지 보수 용이
 - ▣ 기 작성된 코드의 재사용 하여 소프트웨어 개발 기간 단축
 - ▣ 기존 소프트웨어 기능의 수정 및 확장이 쉬움
- 실세계에 대한 쉬운 모델링
 - ▣ 과거의 소프트웨어
 - 수학 계산이나 통계 처리에 편리한 절차 지향 언어가 적합
 - ▣ 현대의 소프트웨어
 - 물체 혹은 객체의 상호 작용에 대한 묘사가 필요
 - 실세계는 객체로 구성된 세계
 - 객체를 중심으로 하는 객체 지향 언어 적합

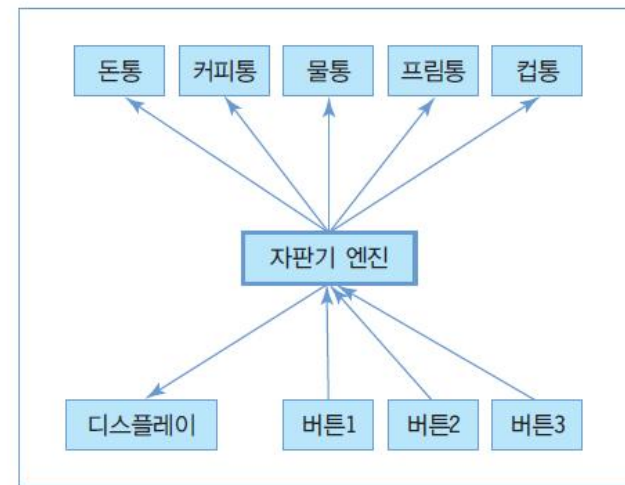
절차 지향 프로그래밍과 객체 지향 프로그래밍

16



(a) 절차 지향적 프로그래밍으로 구현할 때의 흐름도

- 실행하고자 하는 절차대로 일련의 명령어 나열.
- 흐름도를 설계하고 흐름도에 따라 프로그램 작성



(b) 객체 지향적 프로그래밍으로 구현할 때의 객체 관계도

- 객체들을 정의하고, 객체들의 상호 관계, 상호 작용으로 구현

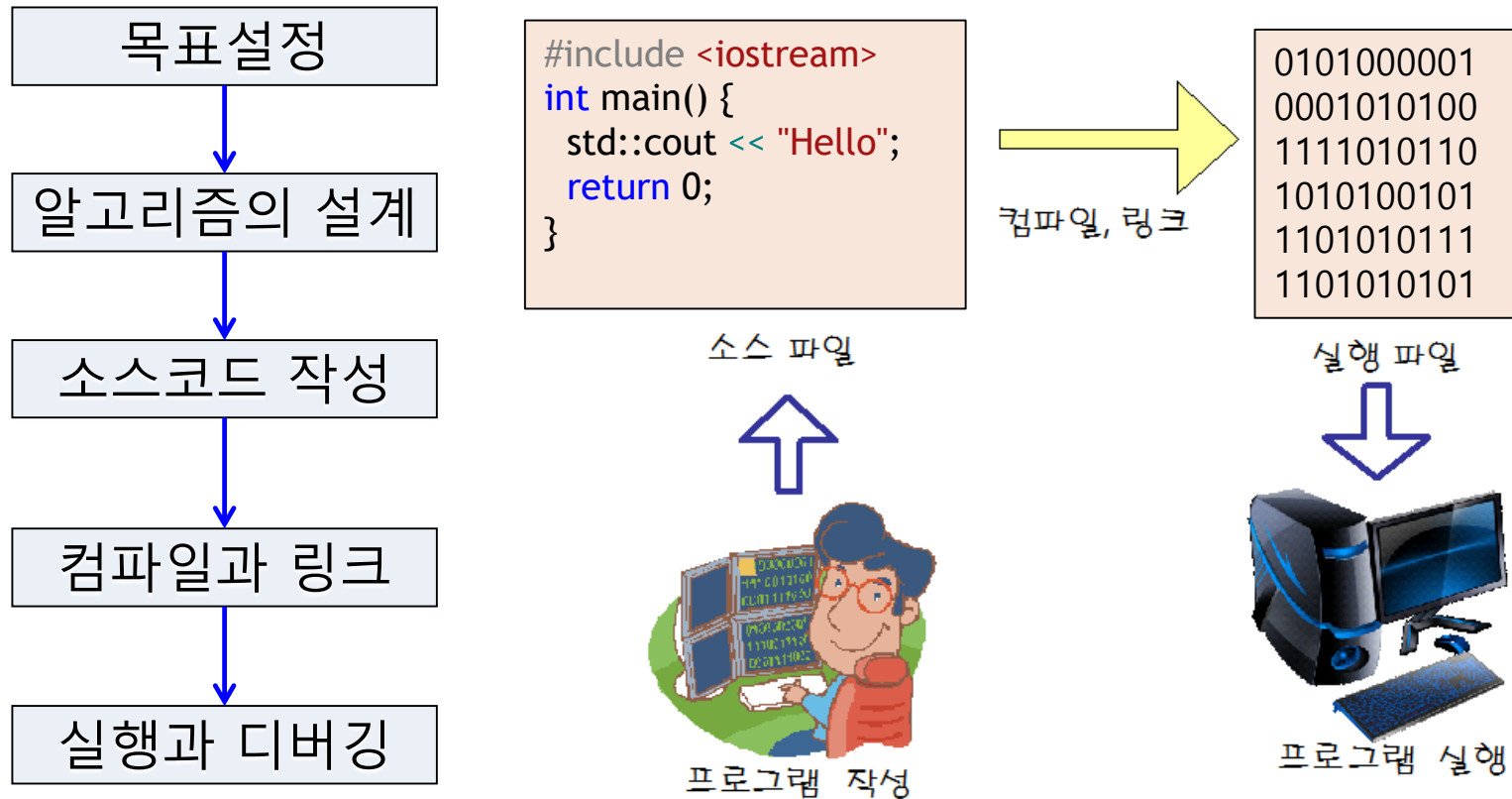
C++ 언어의 아킬레스

17

- C++ 언어는 C 언어와의 호환성 추구
 - ▣ 장점 : 기존에 개발된 C 프로그램 코드 활용
 - ▣ 단점 : 객체지향 언어의 철학이 지켜지지 않는 문제점
 - C++에서 전역 변수와 전역 함수를 사용할 수 밖에 없음
 - 부작용(side effect) 발생 염려

C++ 프로그램 개발 과정

18



알고리즘 -> 문제를 해결하는 절차 또는 방법

C++ 프로그램 개발 과정

19



C++ 소스 프로그램 작성

```
#include <iostream>
int main() {
    std::cout << "Hello";
    return 0;
}
```

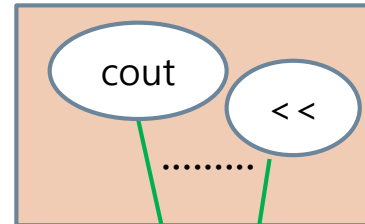
소스 파일
(hello.cpp)

컴파일

```
_main,12#
$<<01010
00000111
_Hello001
```

목적 파일
(hello.obj)

C++ 라이브러리



링크

```
01010000
01000101
01001111
01011010
10100101
11010101
```

실행 파일
(hello.exe)

실행



오류 발생

디버깅

오류 수정



편집 단계

20

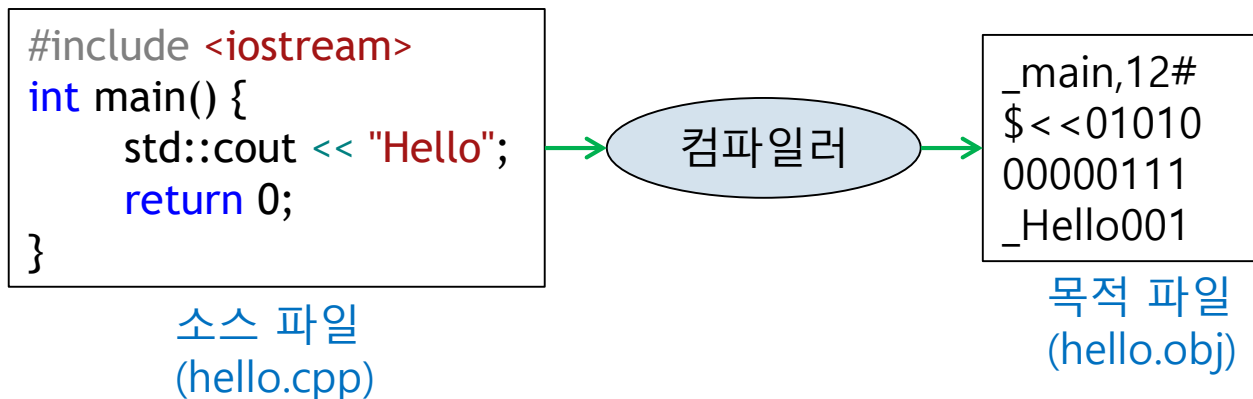
- C언어로 컴퓨터가 처리할 작업을 기술하여 소스 코드를 작성
- **소스 파일(source file)** : C++ 소스 코드가 들어있는 텍스트 파일
- C++ 소스 파일의 표준 확장자는 **.cpp** -> hello.cpp
- 텍스트 **에디터(editor)**를 이용하여 작성

```
#include <iostream>
int main() {
    std::cout << "Hello";
    return 0;
}
```

컴파일 단계

21

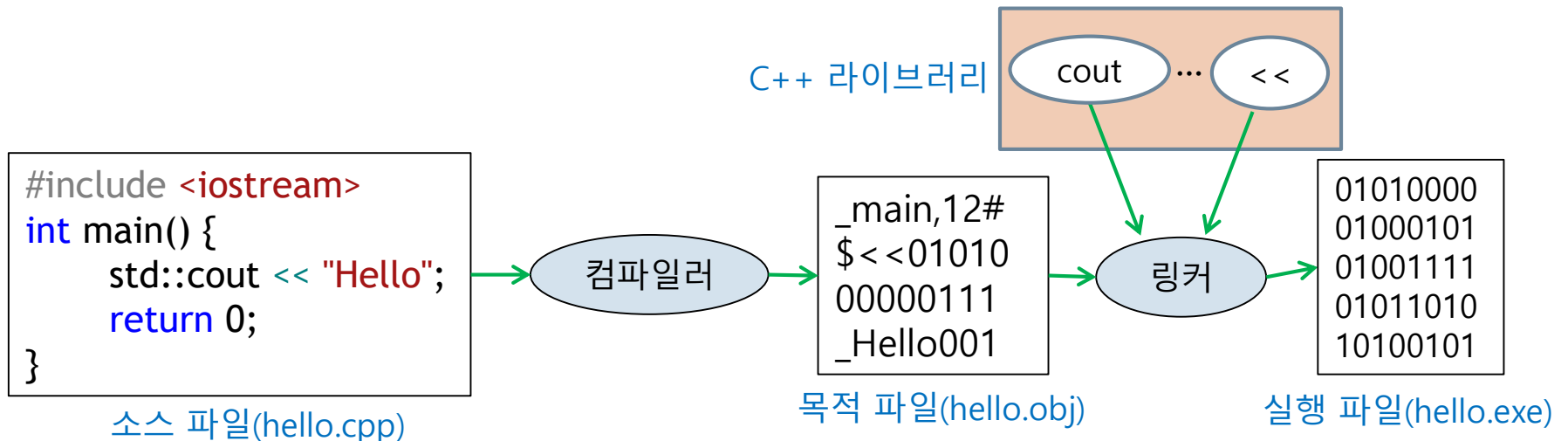
- C++ 소스 코드의 문법을 체크하고 기계어(이진수)로 번역하는 단계
- **목적 파일(object file)** : 기계어로 변환된 파일, 확장자는 **.obj**
-> hello.obj
- 컴파일을 수행하는 프로그램을 **컴파일러(compiler)**라고 부름



링크(링킹) 단계

22

- 목적 파일은 바로 실행할 수 없음
- 목적 파일과 C++ 표준 라이브러리 파일(목적파일)을 연결하여 하나의 실행 파일을 만드는 과정
- **라이브러리(library)**: 프로그램을 만들 때 많이 사용되는 기능을 미리 작성해 놓은 코드(`printf`, `scanf`, `cin`, `cout` 등) -> 목적 파일 형태로 존재
- **실행 파일(executable file)** : 실행이 가능한 기계어로 작성된 파일, 확장자가 **.exe** -> `hello.exe`
- 링크를 수행하는 프로그램을 **링커(linker)**라고 부름

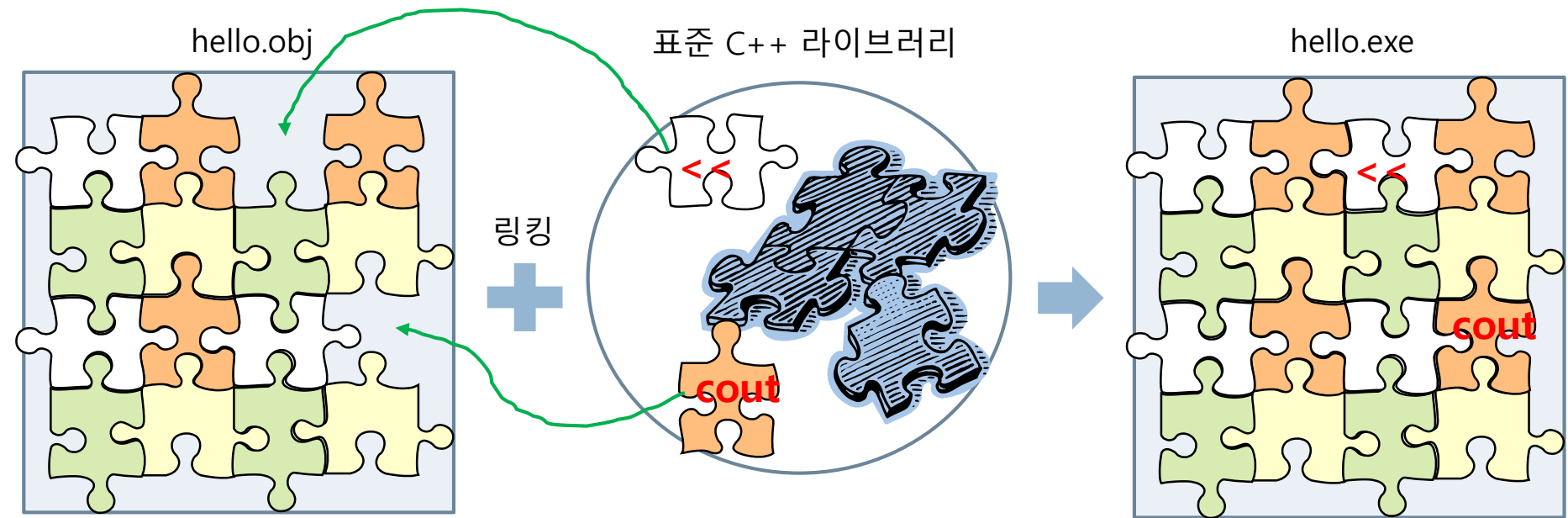


링크(링킹) 단계

23

- 목적 파일과 C++ 표준 라이브러리 파일(목적파일)을 연결

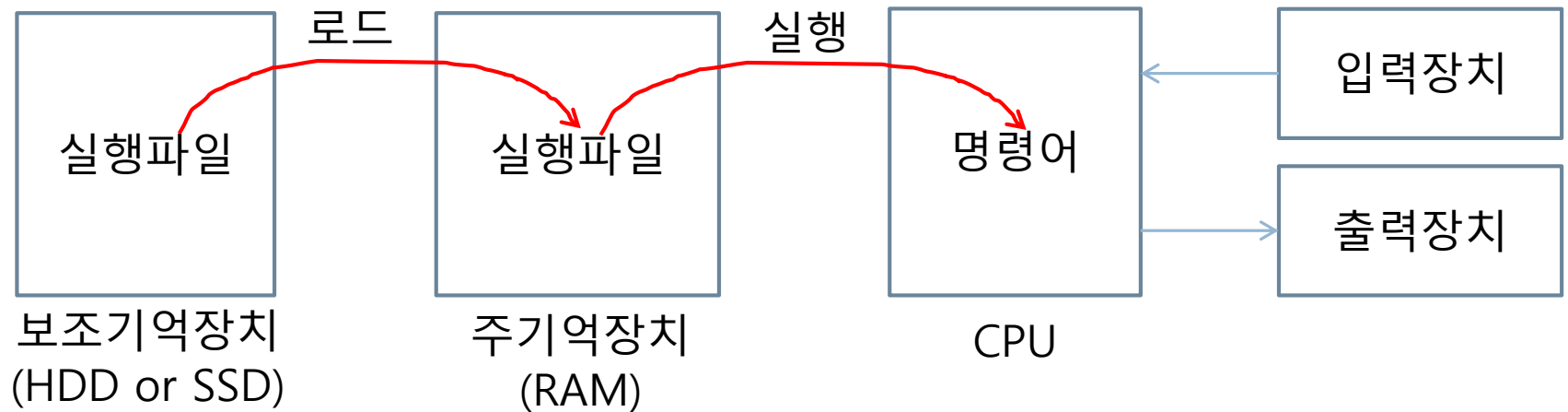
hello.obj + cout 객체 + << 연산자 함수 => hello.exe를 만듦



실행 단계

24

- 실행파일의 내용을 컴퓨터가 수행하는 단계
- 로더(loader) : 실행파일을 주기억장치(RAM)로 로드(load)하고 실행시켜주는 프로그램



디버깅 단계

25

- 버그(bug) : 프로그램에 존재하는 오류(error)
- 디버깅(debugging) : 소스코드에 존재하는 버그를 수정하는 작업
- 디버거(Debugger) : 디버깅을 도와주는 프로그램
- 각 단계별로 오류(컴파일오류, 링크오류, 실행오류)가 발생하면 소스 파일을 수정한 후에 다시 컴파일-> 링크 -> 실행 과정을 반복
- 통합개발환경(IDE) : 프로그램 개발에 필요한 에디터, 컴파일러, 링커, 디버거 등을 통합해 놓은 프로그램, 예) Visual Studio, Eclipse, VS Code 등

정리

26

단계		설명	사용프로그램	결과파일	확장자
편집(Edit)		C++ 언어로 소스파일을 작성하는 단계	에디터(Editor)	소스파일 (source file)	*.cpp
빌드 (Build)	컴파일 (Compile)	소스파일(C++ 언어)을 목적파일(기계어)로 번역하는 단계	컴파일러 (Compiler)	목적파일 (object file)	*.obj
	링크 (Link)	여러 개의 목적파일을 하나의 실행파일로 만드는 과정	링커(Linker)	실행파일 (execute file)	*.exe
실행(Execute)		프로그램을 실행	로더(loader)		
디버깅 (Debugging)		오류(버그)를 수정하는 작업	디버거 (Debugger)		

C++ 표준 라이브러리

27

- C 라이브러리
 - ▣ 기존 C 표준 라이브러리를 수용, C++에서 사용할 수 있게 한 함수들
 - ▣ 이름이 c로 시작하는 헤더 파일에 선언됨
- C++ 입출력 라이브러리
 - ▣ 콘솔 및 파일 입출력을 위한 라이브러리
- C++ STL 라이브러리
 - ▣ 제네릭 프로그래밍을 지원하기 위해 템플릿 라이브러리

algorithm	complex	exception	list	stack
bitset	csetjmp	fstream	locale	stdexcept
cassert	csignal	functional	map	strstream
cctype	cstdarg	iomanip	memory	streambuf
cerrno	cstddef	ios	new	string
cfloat	cstdio	iosfwd	numeric	typeinfo
ciso646	cstdlib	iostream	ostream	utility
climits	cstring	istream	queue	valarray
clocale	ctime	iterator	set	vector
cmath	deque	limits	sstream	

*⟨new⟩ 헤더 파일은 STL에 포함되지 않는 기타 기능을 구현함

실습과제1

28

- 강의노트에서 몰랐던 용어에 대하여 자세히 조사하시오
- 알고리즘 설계와 코딩 중 무엇이 중요하고 그 이유는 무엇인가?
- 클래스와 객체가 무엇인지 설명하시오.
- 절차지향 프로그래밍 언어와 객체지향 프로그래밍 언어의 차이를 조사하라.
- 라이브러리가 무엇인지 자세히 설명하시오.
- 라이브러리가 목적파일 형태로 존재하는 이유를 설명하라.
- 링크과정이 필요한 이유를 설명하라.

과제 제출 방법

29

- 아래 주소의 작성방법대로 github에 제출할 것
 - ▣ <https://github.com/2sungryul/CppProgramming/tree/main>
- 기한 : 과제 게시판에 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목, 날짜, 작성자(학번, 이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```